Tetta AI

Phase 1: Mid-Semester Project (The MVP) - Code Use

STUDENT VIEW

MISY350 • Business Application Development

Attempt 5 • Apr 5, 2026 at 10:53 PM • Apr 5, 2026 at 10:54 PM

90 /100

Well-structured Streamlit app with two roles, clear session-state navigation, robust JSON persistence helpers, and thoughtful UI polish. Core workflows (register/login, booking, rescheduling, cancellation, doctor updates) are implemented end-to-end with validation and user notices. A few Streamlit-specific edge cases and minor organization gaps limit perfection but the submission is a strong, working MVP.

Rubric Overview

Score distribution across all criteria

Multi-page structure and sess…	90%
Login, registration, and authe…	93%
Dual-role architecture and rol…	93%
CRUD functionality and JSO…	95%
Functionality, validation, notic…	90%
Basic layout and usability	90%
Code organization and proce…	80%
Streamlit implementation qual…	80%
Layout effort, polish, and con…	80%

Rubric Breakdown

Multi-page structure and session-state navigation

9 / 10

Page navigation is well-handled via st.session_state.page with navigate_to and st.rerun; role guards and reset_session prevent unauthorized access. Flash messaging and centralized session defaults are implemented thoughtfully. Consider ensuring page state keys are always reset when pages change to avoid stale selections after reruns.

Login, registration, and authentication flow

14 / 15

Patient and doctor registration/login flows work with input validation, duplicate-email checks, hashed passwords, and appropriate routing after login. Error/success notices and redirects are present. For added robustness, consider explicit confirmation emails or stronger password rules as future enhancements.

Dual-role architecture and role-based feature structure

14 / 15

The app cleanly supports Patient and Doctor roles with distinct sidebars, dashboards, and pages, and role routing is enforced on page renderers. Features for each role are meaningful and clearly separated. Consider extracting role-based authorization checks into a small helper to avoid repeated redirect logic.

CRUD functionality and JSON persistence

19 / 20

Create, update, and delete flows (appointments, slots, accounts) persist to JSON and include atomic write helpers and a transactional_save for multi-file updates. Save/read error handling is present and in-memory rollbacks are attempted on failure. One area to tighten is always reloading fresh data immediately before writes in multi-user contexts to further reduce race conditions.

Functionality, validation, notices, and user flow

9 / 10

Workflows are end-to-end with validations, warnings, spinners, success messages, and clear failure handling for most operations. The AI assistant and flash messages improve user guidance. A few UX edge cases (concurrent failure recovery messaging) could be clarified for users.

Basic layout and usability

9 / 10

Layout is planned: sidebar navigation, columns, metrics, and containerized appointment cards yield a readable dashboard-like UI. Custom CSS and badges add polish. Some screens still expose raw data tables (dataframe) but overall the UI reads well and is organized.

Code organization and process separation (Part B)

4 / 5

Code is logically grouped (persistence, validation, business logic, UI) and helper functions are used, which aids readability. However, the single-file layout is large; extracting modules or grouping related functions into classes

would improve separation and maintainability.

Streamlit implementation quality and best practices (Part B)

4 / 5

Good use of st.session_state, keys, st.rerun, and per-item widget keys for doctor notes/status. Be cautious of dynamic selectbox options and keys when underlying lists change — a couple of places could create stale UI or duplicate-key behavior. Consider more conservative use of stable explicit keys for dynamic widgets.

Layout effort, polish, and consistency (Design Excellence — Part C)

8 / 10

Strong visual polish: custom CSS, section headers, slot cards, and badges give the app a consistent, professional feel across pages. Some default Streamlit outputs (dataframe) remain, so a few screens could be further styled to match the custom components for full consistency.

Strengths

- Clear multi-page flow using st.session_state with navigate_to/st.rerun and centralized reset/show flash helpers.
- Complete login and registration flows for patients and doctors with validation and password hashing.
- Distinct Patient and Doctor roles with tailored pages and functionality (booking, dashboards, slot management).
- Robust JSON persistence: transactional_save, atomic writes, and careful error handling for save/load operations.
- Good UX touches: custom CSS, status badges, AI assistant, and informative flash/notice messages throughout.

Needs Attention

- Some Streamlit widget keys derive from appointment IDs but dynamic widget creation (per-appointment keys) could still cause duplicate-key or stale-widget issues when lists change.
- A few places rely on in-memory list mutation before saves; while transactional_save rolls back files, more defensive cloning or explicit reload-before-write would reduce transient race conditions.
- Code is large and mostly in a single file; extracting related groups (persistence, validation, pages) into modules or functions would improve testability and readability.
- Edge cases: concurrent updates (two users) and error recovery around failed transactional_save could present UX confusion in multi-user deployments.